

StockPro

Modélisation des Données NoSQL — MongoDB

Projet	Base de Données Avancées
Département	Informatique
Année	2024 / 2025
Lieu	Libreville, Gabon
Membres	MAVIOGA Jude Tanguy PEMBE MOUPOMA Vinciane Franelle NZOGHE NKOGHE Abénislain Yannick

1. Présentation des collections

La base de données **gestionStock** contient cinq collections principales. Ce document décrit leur structure, les relations entre elles, et justifie les choix techniques d'embedding et de referencing adoptés pour le projet StockPro.

2. Collection : articles

Stocke le catalogue de tous les produits gérés. Chaque document représente une référence produit unique avec son prix unitaire et son seuil d'alerte.

```
// Collection : articles
{
  "_id"      : ObjectId,          // Identifiant unique MongoDB
  "nom"      : String,           // Ex: "Sac de riz 25kg"
  "reference" : String,           // Ex: "RIZ-25KG" (UNIQUE)
  "prix"     : Number,           // Prix unitaire en FCFA
  "seuilAlerte": Number,         // Seuil declenchement alerte
  "createdAt" : Date,
  "updatedAt" : Date
}
```

Champ	Type	Contrainte	Description
_id	ObjectId	Requis, unique	Identifiant auto-généré
nom	String	Requis	Nom commercial du produit
reference	String	Requis, unique	Code référence article
prix	Number	Requis, > 0	Prix unitaire FCFA
seuilAlerte	Number	Requis	Seuil bas déclenchant l'alerte
createdAt	Date	Auto	Horodatage création
updatedAt	Date	Auto	Horodatage dernière MAJ

3. Collection : entrepots

Représente chaque site de stockage. Le champ **stock** est un tableau embarqué (pattern *embed*) qui contient directement les quantités par article. Ce choix évite des jointures répétées lors de la lecture du stock en temps réel.

```
// Collection : entrepots
{
  "_id"      : ObjectId,
  "nom"      : String,          // Ex: "Entrepot Principal Libreville"
  "localisation": String,       // Ex: "Boulevard Triomphal, Libreville"
  "stock"    : [               // EMBEDDED - tableau de stock

```

```
{
  "_id"      : ObjectId,
  "article" : ObjectId,      // Ref vers articles._id
  "quantite": Number
}
],
"createdAt" : Date,
"updatedAt" : Date
}
```

4. Collection : mouvements

Assure la traçabilité complète de tous les flux de stock. Chaque document capture le contexte complet au moment de l'opération : qui, quoi, quand, d'où, vers où. Les champs *fournisseur*, *source*, *destination* et *destinataire* sont conditionnels selon le type de mouvement.

```
// Collection : mouvements
{
  "_id"      : ObjectId,
  "type"     : String,      // "ENTREE" | "SORTIE" | "TRANSFERT"
  "article"  : ObjectId,    // REF -> articles._id
  "quantite" : Number,
  "utilisateur": ObjectId,  // REF -> users._id (QUI)
  "date"     : Date,        // QUAND
  "createdAt" : Date,
  // Champs conditionnels :
  "fournisseur": ObjectId,  // REF -> fournisseurs._id (ENTREE)
  "destination": ObjectId, // REF -> entrepots._id (ENTREE + TRANSFERT)
  "source"     : ObjectId,  // REF -> entrepots._id (SORTIE + TRANSFERT)
  "destinataire": String    // Nom client/service (SORTIE)
}
```

Question	Champ MongoDB	Type
QUI ?	utilisateur	ObjectId → users
QUOI ?	article + quantite	ObjectId + Number
QUAND ?	createdAt	Date (auto)
D'OÙ ?	source / fournisseur	ObjectId → entrepots / fournisseurs
VERS OÙ ?	destination / destinataire	ObjectId / String

5. Collection : fournisseurs

```
// Collection : fournisseurs
{
  "_id"      : ObjectId,
  "nom"      : String,      // Raison sociale
  "contact"  : String,      // Nom du responsable
  "email"    : String,
  "telephone": String,
  "createdAt": Date,
  "updatedAt": Date
}
```

6. Collection : users

```
// Collection : users
{
  "_id"      : ObjectId,
  "nom"      : String,
  "email"    : String,          // Unique
  "motDePasse" : String,        // Hashe avec bcrypt
  "role"     : String,          // "ADMIN" | "USER"
  "isActive" : Boolean,
  "sessionId" : String | null,
  "createdAt" : Date,
  "updatedAt" : Date
}
```

7. Relations entre collections

Relation	Type	Cardinalité	Description
entrepots.stock.article → articles._id	Référence (embed)	N:1	Chaque ligne de stock référence un article
mouvements.article → articles._id	Référence	N:1	Chaque mouvement concerne un article
mouvements.destination → entrepots._id	Référence	N:1	Entrepôt destination (ENTREE + TRANSFERT)
mouvements.source → entrepots._id	Référence	N:1	Entrepôt source (SORTIE + TRANSFERT)
mouvements.fournisseur → fournisseurs._id	Référence	N:1	Fournisseur (ENTREE uniquement)
mouvements.utilisateur → users._id	Référence	N:1	Utilisateur auteur de l'opération

8. Choix techniques : Embedding vs Referencing

Relation	Stratégie	Justification
Entrepôt → Stock	EMBED	Le stock est toujours lu avec l'entrepôt. L'embedding évite un \$lookup et permet la mise à jour atomique.
Mouvement → Article	REFERENCE	Les articles sont partagés entre de nombreux mouvements. La duplication serait coûteuse.
Mouvement → Entrepôt	REFERENCE	L'entrepôt est une entité indépendante, réutilisée dans de nombreux contextes. La référence permet de lier les mouvements à l'entrepôt sans duplication.
Mouvement → Utilisateur	REFERENCE	Traçabilité sans dupliquer les données utilisateur. Le nom ou le rôle peut évoluer sans impact sur les mouvements.
Mouvement → Fournisseur	REFERENCE	Même logique : entité indépendante partagée entre de nombreuses entrées.

9. Justification des choix

- **Flexibilité du schéma** : MongoDB permet d'ajouter des attributs variables selon la nature de l'article (poids, dimensions, date de péremption) sans ALTER TABLE.
- **Documents imbriqués naturels** : le tableau stock[] d'un entrepôt correspond directement à la réalité métier et évite des JOIN complexes lors de la consultation du stock en temps réel.
- **Pipeline d'agrégation** : la requête d'alertes stock (articles sous seuil par entrepôt) s'exprime lisiblement avec \$unwind / \$lookup / \$match.
- **Transactions ACID** depuis MongoDB 4.0 : le risque d'incohérence lors des transferts inter-entrepôts est résolu nativement par les sessions multi-documents.
- **Stack homogène** : Node.js + Mongoose + MongoDB manipule du JSON de bout en bout, réduisant la friction de conversion de types.